

roles.yml: Project Access Token

The snowflakeprovisioningrolesyml CI job runs updaterolesyml.py which updates roles.yml file.

The changes to roles.yml within the CI job are pushed back to the branch/MR.

In order to push to the repo from within the CI pipeline, a Project Access Token (PAT) is needed, more info pushing to the remote repo in this [StackOverflow](#) answer.

The PAT is named snowflakeprovisioningautomation and was created in the 'GitLab Data Team' project, using the analyticsapi@gitlab.com account.

The PAT value is saved within 1Pass, and also as a CI environment variable so that it can be used by the GitLab runner.

snowflakeusers.yml - end of file issue

When adding a user to the snowflakeusers.yml file, specifically when appending to the bottom of the file, it causes unexpected behavior if done using the GitLab Single File Editor, more info in this [issue](#).

The workaround is that at the bottom of snowflakeusers.yml, it has this comment:

```
yml
do not insert users below this line
```

Snowflake Deprovisioning Users

Inactive Snowflake users will be deprovisioned weekly via snowflakecleanup DAG, implemented in this [issue](#).

All active Snowflake users/roles are declared within roles.yml. Therefore, if any users in Snowflake are missing within roles.yml, they are considered inactive and the process will drop them.

These users will be dropped by running the following deprovisionuser.sql script.

This process is not exposed via CI job due to its sensitive nature and because it is less time sensitive. Therefore, a weekly 'cleanup' task via Airflow will be run instead.

Snowflake user/service account

The permifrostbotuser is used to run both Snowflake provisioning and deprovisioning processes. This is for 2 reasons:

1. permifrostbotuser already has the proper permissions to run provisioning/deprovisioning as the same perms are needed to run existing Permifrost jobs.
1. The permifrostbotuser already runs existing Permifrost jobs using both Airflow and GitLab CI, so the applied NSP IP addresses will not be redundant when added for both provisioning (run

via CI) /deprovisioning (run via Airflow).

Provisioning permissions to external tables to user roles

Provisioning USAGE permissions for external tables to user roles inside snowflake is not handled by permifrost in the moment. If you have to provision access for an external table to a user role, then it must be granted manually via GRANT command in snowflakedocs using a securityadmin role. This implies that the user role already has access to the schema and the db in which the external table is located, if not add them to the roles.yml.

Logging in and using the correct role

When you apply for a Snowflake account via an AR and get access provisioned it takes until 3.00AM UTC for the change to take effect. This is because we have a script running daily to provision the access in Snowflake. When you can login, you can do this via Okta. After you logged in via Okta, you need to select the right role that is attached to your account. This is by default the same as your account and it follows the convention of your email address minus @gitlab.com.

When you don't select the right role in Snowflake, you only see the following Snowflake objects:

Selecting the right role can be done via the GUI.
When in Snowsight home screen, in the up left corner.

1. Click on the arrow near your name
2. Select Switch Role
3. Select your role

When in Snowsight in a worksheet, in the up right corner.

1. Click on public
2. Select your role

You can set this to your default by running the following:

```
ALTER USER <YOURUSERNAME> SET DEFAULTROLE = '<YOURROLE>'
```

Compute Resources

Compute resources in Snowflake are known as "warehouses".
To use our credit consumption effectively, we try to minimize the amount of warehouses. For development purposes (executing dbt jobs locally, running MR pipelines and querying in Snowflake) we use the devx warehouse. The names of the warehouse are appended with their size

(devxs for extra small).

warehouse	purpose
max query (minutes)	

admin	This is for permission bot and other admin tasks
10	
dataclassification	This is for running the data classification and labelling process in Snowflake
60	
devxs/m/l/xl	This is used for development purposes, to be used when using the Snowflake UI and in CI-pipelines
180	
gainsightxs	This is used for gainsight data pump
30	
gitlabpostgres databases	This is for extraction jobs that pull from GitLab internal Postgres databases
10	
grafana	This is exclusively for Grafana to use
60	
loading	This is for our Extract and Load jobs and testing new Meltano loaders
120	
reporting	This is for the BI tool for querying.
30	
transformingxs	These are for production dbt jobs
180	
transformings	These are for production dbt jobs
180	
transformingl	These are for production dbt jobs
240	
transformingxl	These are for production dbt jobs
180	
transforming2xl	For refreshing Snowplow models
120	
transforming4xl	This is for the Airflow dag: dbtfullrefresh
120	
usageping	This is used for the serviceping and servicepingbackfill load.
120	

If you're running into query time limits please check your query for optimisation. A bad performing query in development will result in a bad performing query in production, having impact on a daily basis. Please always use the right (size) warehouse. Ground rules of using/selecting a warehouse:

Warehouses are set as t-shirt sizes. Larger warehouses are more costly for GitLab
Consider using a running warehouse

If you resume a paused warehouse, there is a initial start cost

Every warehouse suspends after a set period, but when idle (time between query result and the suspend time), we still consume snowflake credits

In general we don't spend more money if we run concurrent queries.

The query timeout in Snowflake is set to 30 minutes for the REPORTING warehouse.

Data Storage

We use three primary databases: raw, prep, and prod.

The raw database is where data is first loaded into Snowflake; the other databases are for data that is ready for analysis (or getting there).

{{% alert color="warning" %}}

All tables and views in prep and prod are controlled (created, updated) via dbt. Every Quarter the Data Platform Team runs a check for tables and views that are not related to a dbt model and will be removed.

{{% /alert %}}

The following list of schema are exceptions and not checked:

SNOWFLOW%
DOTCOMUSAGEEVENTS%
INFORMATIONSCHEMA
BONEYARD
TDF
CONTAINERREGISTRY
FULLTABLECLONES
QUALTRICSMAILINGLIST
NETSUITEFIVETRAN

There is a snowflake database, which contains information about the entire GitLab instance. This includes all tables, views, queries, users, etc.

There is a covid19 database, which is a shared database managed through the Snowflake Data Exchange.

There is a testingdb database, which is used for testing Permifrost.

There is a bitoleval database, which is used for testing bi tooling. Users are able create own testing sets manually.

All databases not defined in our roles.yml Permifrost file are removed on a weekly basis.

Database	Suitable to use in Tableau
raw	No
prep	No
prod	Yes

Only the prod database should be used in Tableau as this data has been transformed and modeled for business use. Using raw and prep databases in Tableau could result in incorrect data and or broken queries/dashboards now or in the future. Important to keep in mind that data transformations are checked and tested only for the prod database results. This means if dashboards are directly connected to the raw or prep database it could break or report wrong

data.

Raw

No dbt models exist for this data and so it may be the case that the data needs review or transformation in order to be useful or accurate. This review, documentation, and transformation all happens downstream in dbt for PREP and PROD. This database should not be used in Tableau.

Raw may contain sensitive data, so permissions need to be carefully controlled

RAW will contain data that isn't ready for business use.

Data is stored in different schemas based on the source

User access can be controlled by schema and tables

Snowflake Data Share

Snowflake data sharing enables sharing various Snowflake objects like databases, tables, secure views, and a couple more from one Snowflake account to another. Snowflake shares can be both Inbound and outbound. Inbound share, which is being used at Gitlab, is for accessing third-party data sources like ZoomInfo and Zuora Revenue; the mechanism followed here is direct share, where data providers share specific database objects to our Snowflake account. Outbound sharing is when we want to share our data with a third party. This involves creating an outbound share of a snowflake object in their account and granting access to the snowflake object (table, view, database, etc.) that needs to be shared to an external account using either a web interface or SQL.

Snowflake Data Shares can be seen as an extension of the raw layer, but sharded (and) in different accounts. We don't see Snowflake Data Shares as a source from which data needs to be copied, but rather we connect directly to Snowflake Data Shares as we do to the raw layer (i.e., with dbt). This approach helps avoid creating extra processes and makes the pipeline more efficient.

Prep

This is the first layer of verification and transformation in the warehouse, but is not yet ready for general business use. This database should not be used in Tableau.

Source models are built in logical schemas corresponding to the data source (i.e. sfcdc, zuora)

PREPARATION - this is the default schema where dbt models are built

SENSITIVE

Prod

This database and all schemas and tables in it are queryable by Tableau. This data has been transformed and modeled for business use.

With the exception of public, and boneyard, all schemas are controlled by dbt. See the dbt guide for more information.

Folder Structure in Analytics Project

The table below shows a mapping of how models stored within folders in the models/ directory in the analytics project will be materialized in the data warehouse.

The source of truth for this is in the dbtproject.yml configuration file.

Folder in snowflake-dbt/models/	db.schema	Details	Queryable in Tableau	
- - - :-				
common/	prod.common	Top-level folder for facts and dimensions. Do not put models here.		
Yes				
common/bridge	prod.common	Sub-folder for creating many-to-many mappings between data that come from different sources.	Yes	
common/dimensionslocal	prod.common	Sub-folder with directories containing dimensions for each analysis area.	Yes	
common/dimensionsshared	prod.common	Sub-folder with dimensions that relate to every analysis area.	Yes	
common/factsfinancial	prod.common	Sub-folder with facts for the financial analysis area.		
Yes				
common/factsproductandengineering	prod.common	Sub-folder with facts for the product and engineering analysis area.	Yes	
common/factssalesandmarketing	prod.common	Sub-folder with facts for the sales and marketing analysis area.	Yes	
common/sensitive/	prep.sensitive	Facts/dims that contain sensitive data.	No	
commonmapping/	prod.commonmapping	Used for creating one-to-one mappings between data that come from different sources.	Yes	
commonmart/	prod.commonmart	Joined dims and facts that are relevant to all analysis areas.	Yes	
commonmartfinance/	prod.commonmart	Joined dims and facts that are relevant to finance.		
Yes				
commonmartmarketing/	prod.commonmart	Joined dims and facts that are relevant to marketing.	Yes	
commonmartproduct/	prod.commonmart	Joined dims and facts that are relevant to product.		
Yes				
commonmartsales/	prod.commonmart	Joined dims and facts that are relevant to sales.	Yes	
commonprep/	prod.commonprep	Preparation tables for mapping, bridge, dims, and facts.		
Yes				
marts/	varies	Contains mart-level data and data pumps that send data to third party sources.	Yes	
legacy/	prod.legacy	Contains models built in a non-dimensional manner	Yes	
sources/	prep.source	Contains source models. Schema is based on data source	No	
workspaces/	prod.workspaceworkspace	Contains workspace models that aren't subject to SQL or dbt standards.	Yes	
common/restricted	prod.restricteddomaincommon	Top-level folder for restricted facts and dimensions. Equivalent of the regular common schema, but for restricted data.	Yes	
commonmapping/restricted	prod.restricteddomaincommonmapping	Contains restricted mapping, bridge, or look-up tables. Equivalent of regular common mapping schema, but for restricted data.	Yes	
marts/restricted	prod.restricteddomaincommonmarts		Yes	
legacy/restricted	prod.restricteddomainlegacy	Contains restricted models built in a non-dimensional manner. Equivalent of the normal legacy schema, but for restricted data.	Yes	

Static

For data warehouse use cases that require us to store data for our users without updating it automatically with dbt we use the STATIC database. This also allows for analysts and other users to create their own data resources (tables, views, temporary tables). There is a sensitive schema for sensitive data within the static database. If your use case for static requires the use or storage of sensitive data please create an issue for the data engineers.

Scenario's we have been using the STATIC database:

A request comes in to upload a set of data into one of our data sources.
This set of data is going to be uploaded once and never updated again.

In this case we have created a new table in the STATIC database and loaded the data there via BULK UPLOAD / COPY command. Then this model has been exposed to the PREP layer. The final model reads from this table via a UNION statement.

This way we have the data in the STATIC database and even if we perform a full-refresh of the data source, we will be able to include this manually uploaded set of records.

Examples of this implementation can be found below:

Qualtrics, [Link to the MR](#)

Clari, [Link to the MR](#)

Data Masking

We use data masking obfuscate private or sensitive information with our data warehouse. Masking can be applied in a dynamic or static manner depending on the particular data needs. Masking can be applied at the request of the data source system owner or at discretion of the Data Team. As our current data masking methods are applied procedurally using dbt they can only be applied in the PREP and PROD database. If masking is required in the RAW database alternant methods of masking should be investigated.

Static Masking

Static data masking is applied during the transformation of the data and the masked result is materialized into the table or view. This will mask the data for all users regardless of role or access permission. This is accomplished in the code with tools such as the `hashsensitivecolumns` macro within dbt.

Dynamic Masking

Dynamic masking is currently applied on tables or views in the prep and prod layer at query run time based on assigned policies and user roles using the Dynamic Data Masking capabilities of Snowflake. Dynamic masking allows for data to be unmasked for selected users while masked for all other users. This is accomplished by creating masking policies that are then applied to the column at the time of table or view creation. Masking policies are maintained within the data warehouse source code repository. Please see the dbt guide to setup dynamic masking.

Note: Dynamic masking is not applied on raw database yet.

Timezones

All timestamp data in the warehouse should be stored in UTC. The default timezone for a Snowflake sessions is PT, but we have overridden this so that UTC is the default. This means that when `currenttimestamp()` is queried, the result is returned in UTC.

Stitch explicitly converts timestamps to UTC. Fivetran does this as well (confirmed via support chat).

The only exception to this rule is the use of pacific time to create `dateid` in fact tables, which should always be created by the `getdateptid` dbt macro and labeled with the `ptid` suffix.

Snapshots

We use the term snapshots in multiple places throughout the data team handbook and the term can be confusing depending on the context. Snapshots as defined by the dictionary is "a record of the contents of a storage location or data file at a given time". We strive to use this definition whenever we use the word.

dbt

The most common usage is in reference to dbt snapshots. When `dbt snapshots` is run, it takes the current state of the source data and updates the corresponding snapshot table, which is a table that contains the full history of the source table. It has `validto` and `validfrom` fields indicating the time period for which that particular snapshot is valid. See the dbt snapshots section in our dbt guide for more technical information.

The tables generated and maintained by dbt snapshots are the raw historical snapshot tables. We will build downstream models on top of these raw historical snapshots for further querying. The `snapshots` folder is where we store the dbt models. One common model we may build is one that generate a single entry (i.e. a single snapshot) for a given day; this is useful when there are multiple snapshots taken in a 24 hour period. We also will build models to return the most current snapshot from the raw historical table.

Other uses

Our Greenhouse data can be thought of as a snapshot. We get a daily database dump provided by Greenhouse that we load into Snowflake. If we start taking dbt snapshots of these tables then we would be creating historical snapshots of the Greenhouse data.

The extracts we do for some yaml files can also be thought of as snapshots. This extraction works by taking the full file/table and storing it in its own, timestamped row in the warehouse. This means we have historical snapshots for these files/tables but these are not the same kind of snapshot as dbt. We'd have to do additional transformations to get the same `validto` and `validfrom` behavior.

Language

Snapshot - The state of data at a specific point in time

Take a snapshot - Run the job that takes the state of the data currently and stores it. Can be used in the dbt context. Not recommended to reference our yaml extract jobs - these would be "run the extract".

Historical snapshots - A table that contains data for a given source table at multiple points in time. Most commonly used to reference dbt-generated snapshot tables. Can also be used to reference the yaml extract tables.

Latest snapshot - The most current state of the data we have stored. For dbt snapshots these are the records that have null for the validto. For yaml extracts this correspond to the last time the extraction job was run. For Greenhouse raw, this represents the data as it is in the warehouse. Were we to start taking snapshots of the Greenhouse data the speaker would have to clarify if they mean the raw table or the latest record in the historical snapshots table.

Backups

The scope of data backups at Data Platform level is to ensure data continuity and availability for reporting and analytics purposes. In case of an unforeseen circumstance happening with our data in Snowflake or with our Snowflake platform, the GitLab data team is able to recover and restore data to the desired state. In our backup policy we tried to find a balance between the risk of an unforeseen event and the impact of the mitigated solution.

Note: the (Snowflake) Data Platform doesn't act as a data archival solution for upstream source systems i.e. for compliance reasons. The Data Platform relies on data that was and is made available in upstream source systems.

Unforeseen circumstances

We've identified currently 2 types of unforeseen circumstances:

Incorrect events happening inside the data platform.
Unavailability of the Snowflake environment.

Incorrect events happening inside the data platform

This can be data manipulation action done by a GitLab Team member or by services with access to the data in Snowflake. Some examples are accidentally dropping/truncating a table or running incorrect logic in a transformation.

The vast majority of data in snowflake is copied or derived from copies of our data sources, which is all managed idempotently with dbt and so the most common procedure for data restoration or recovery is through recreating or refreshing objects using dbt Full Refresh. For data in the RAW database, which comes from our extraction pipelines we follow the appropriate Data refresh procedure.

However, there are some exceptions to this. Any data in snowflake which is not a result of idempotent processes or that cannot be refreshed in a practical amount of time should be backed up. For this we use Snowflake Time travel. Which includes:

1. Storage in permanent (not transient) tables.
1. A data retention period of 30 days.

The data retention period is set via dbt This should be implemented in code via a dbt post-hook example.

The following set of rules and guidelines applies to backing up data/using time travel:

It is the responsibility of the CODEOWNER to ensure that the backup processes has been correctly implemented for the data that their code builds or maintains.

Backups (via Time Travel) need not be applied on dbt models by default since these are idempotent and this would result in a huge increase of the storage costs in Snowflake.

The retention period is set to 30 days.

At the moment the following snowflake objects are considered in scope for Time Travel recovery:

RAW.SNAPSHOTS.

Once a table is permanent with a retention period we are able to use Time Travel (internal runbook) in the event we need to recover one of these tables.

Unavailability of the Snowflake environment

For the unlikely event that Snowflake becomes unavailable for an undetermined amount of time, we additionally backup the any business critical data, where Snowflake is the primary source, to Google Cloud Storage (GCS). We execute these backup jobs using dbt's run-operation capabilities. Currently, we backup all of our snapshots daily and retain them for a period of 60 days (per GCS retention policy). If a table should be added to this GCS backup procedure it should be added via the backup manifest.

Snowflake Admin tasks

In order to keep Snowflake up and running, we perform administrative work.

Create new Snowflake external stage for GCS storage bucket

In order for Snowflake to access the files in GCS bucket, the files must be copied into a Snowflake external stage.

To create the external stage, the new path to the bucket must be included (included means appended to the existing list of storage locations) in the STORAGEALLOWEDLOCATIONS attribute. If it is not appended, but overwritten to the existing attributes, all existing storage locations will be erased and stop many pipelines to run. Follow these instructions to append the new external stage:

The GCSINTEGRATION is Snowflake storage integration for gitlab-analysis project in GCP. If the bucket is in different project, a new integration would need to be created.

1. use role ACCOUNTADMIN, if you don't have access to this role, you cannot proceed
1. get all current storage locations by running this:

```
sql
DESC INTEGRATION GCSINTEGRATION;
```

1. From the output, copy the value under propertyvalue where property=STORAGEALLOWEDLOCATIONS
It will look something like: gcs://postgrespipeline/,gcs://snowflakebackups,...

1. Update the Storage Integration, instructions:

take the 'currentpaths' that you just copied and combine it with the 'newpath' that you want to add.

Each path needs to be separated by a ,

Each path needs to have it's own pair of ", These need to be added manually

ALTER statement template:

```
sql
ALTER STORAGE INTEGRATION GCSINTEGRATION
SET STORAGEALLOWEDLOCATIONS = ('currentpath1','currentpath2','newpath');
```

ALTER statement example:

```
sql
ALTER STORAGE INTEGRATION GCSINTEGRATION
SET STORAGEALLOWEDLOCATIONS=
('gcs://postgrespipeline/','gcs://snowflakebackups','gcs://snowflakeexports');
```

1. After you run the ALTER statement, the new stage can now be created, like so:

```
sql
CREATE STAGE "RAW"."PTO".ptoload
STORAGEINTEGRATION = GCSINTEGRATION URL = 'bucket location';
```

Create new Snowflake external stage for AWS S3 storage bucket

This guide explains how to grant Snowflake access to a new S3 bucket using the existing Snowflake storage integration.

Overview

The process involves:

1. Creating a new S3 bucket using terraform
1. Updating the IAM policy to allow Snowflake access to this bucket
1. Updating the Snowflake storage integration configuration

Prerequisites

Access to config-mgmt repo, specifically the aws-gitlab-analysis environment.
Snowflake account access with ACCOUNTADMIN role

Detailed Steps

<details><summary>Click to expand</summary>

1. Create the S3 Bucket

1. In the repository: gitlab-com/gl-infra/config-mgmt

1. Create a new S3 bucket via Terraform in the aws-gitlab-analysis environment:

```
terraform
resource "aws_s3_bucket" "somenewbucket" {
  bucket = "your-new-bucket-name"
  Add other configuration as needed
}
```

2. Update the IAM Policy

1. In the same repo as the previous step, navigate to the policy file in GitLab:

File path: environments/aws-gitlab-analysis/templates/iampolicysnowflakes3integration.json

1. Add the new bucket path under Resource array in the same pattern as of existing bucket.

```
json
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:GetObjectVersion",
    "s3:PutObject",
    "s3:ListBucket"
  ],
  "Resource": [
    "arn:aws:s3:::your-new-bucket-name/",
    "arn:aws:s3:::your-new-bucket-name"
  ]
}
```

1. Just like any change in config-mgmt repo, get approvals, and then run atlantis apply to deploy the change

3. Update the Snowflake Storage Integration

Add the new bucket to the allowed storage locations in Snowflake:

1. Use ACCOUNTADMIN role

1. Update the Snowflake storage integration, be sure you append the new bucket to the existing list of buckets:

```
sql
ALTER STORAGE INTEGRATION S3DATAPUMP
```

```
SET STORAGEALLOWEDLOCATIONS = ('s3://existing-bucket-1/', 's3://existing-bucket-2/',  
's3://your-new-bucket-name/');
```

1. Verify the integration settings:

```
sql  
DESC INTEGRATION S3DATAPUMP;
```

Note: We are treating the S3DATAPUMP Snowflake storage integration as the generic one which is responsible for establishing connection to S3 in the main AWS project where Snowplow instance is running. If we have a new bucket in different project, such as in a customer provided one, we would need to create a new Snowflake integration for that AWS project, Snowflake docs.

4. Verification

To verify everything is working correctly:

1. In Snowflake, attempt to create an external stage using the new bucket
1. Test reading from and writing to the bucket using Snowflake queries

</details>

<i class="fas fa-cogs fa-fw -text-orange"></i>Transformation

We use dbt for all of our transformations.

See our dbt guide for more details on why and how we use this tool.

<i class="fas fa-check-double fa-fw -text-purple"></i>Trusted Data Framework

Data Customers expect Data Teams to provide data they can trust to make their important decisions. And Data Teams need to be confident in the quality of data they deliver. But this is a hard problem to solve: the Enterprise Data Platform is complex and involves multiple stages of data processing and transformation, with tens to hundreds of developers and end-users actively changing and querying data 24 hours a day. The Trusted Data Framework (TDF) supports these quality and trust needs by defining a standard framework for data testing and monitoring across data processing stages, accessible by technical teams and business teams. Implemented as a stand-alone module separate from existing data processing technology, the TDF fulfills the need for an independent data monitoring solution.

Enable everyone to contribute to trusted data, not just analysts and engineers

Enable data validations from top to bottom and across all stages of data processing

Validate data from source system data pipelines

Validate data transforms into dimensional models

Validate critical company data

Deployable independently from central data processing technology

Key Terms

Assertion or Test Case - An individual test and the smallest unit of a test that can be performed. In TDF the test case is expressed either as a SQL statement or via a YAML configuration within SQL-compilation tool, dbt.

Data Schema - The tables, columns, views, and other structural elements that make up a data subject area, create using SQL Data Definition Language (DDL).

Monitoring - Tracking the results of tests cases to help ensure data is ready for use.

Trusted Data Components

The primary elements of the TDF include:

1. A Virtuous Test Cycle that embeds quality as a normal part of daily data development, ranging from new data solutions to break-fix issue resolution.
1. Test Cases Expressed As SQL and YAML which can be developed by anyone.
1. The Trusted Data Schema saves test results for monitoring and alerting, and long-term analysis towards the path of developing wisdom around business processes and data platform performance.
1. Schema-to-Golden Record Coverage to provide broad coverage of the data warehouse domain, ranging from schema to critical "Golden" data.
1. The Trusted Data Dashboard, a business-friendly dashboard to visualize overall test coverage, successes, and failures.
1. The Test Run is when a Test Cases are executed.
1. Row Count test to reconcile the amount of rows between source system and Snowflake

Virtuous Test Cycle

The TDF embraces business users as the most important participant in establishing trusted data and uses a simple and accessible testing model. With SQL and YAML as a test agent, a broad group of people can contribute test cases. The test format is straightforward with simple PASS/FAIL results and just four test case types. Adoption grows quickly as TDF demonstrates value:

Data Customers and Business Users learn the testing framework and create tests themselves
Teams embrace testing as a valuable activity to include at all times, not as a last-minute activity

The Data Team learns to add new tests as part of production-down retrospectives to more rapidly identify issues before they become large problems

Teams develop operational rhythms to continually develop new tests and expand test coverage

Over time, it is not uncommon to develop hundreds of tests cases which are run on a daily basis, continually validating data quality.

Test Cases Expressed As SQL and YAML

SQL is the universal language in databases and nearly everyone who works with data has some level of SQL competency. However, not everyone may be familiar with SQL and we don't want that to limit who can contribute. We use dbt to support the TDF which enables the defining of tests via SQL and YAML.

Trusted Data Schema

With all tests being run via dbt, storing test results is simple. We store the results of every test run in the data warehouse. Storing test results enables a variety of valuable features, including:

- data visualization and pattern analysis test results (total tests run by date, PASS/FAIL rate by subject area, and so on)
- measurement of test coverage over a data subject or schema (number of tests by area)
- measurement of system quality improvements over time (an increase in the PASS rate)
- development of an alerting system based on test result

These test results are parsed and are available for querying in Tableau.

The schema we store all test results is: WORKSPACEDATA.

Note: This schema only contains views.

Schema To Golden Record Coverage

The Data Warehouse environment can change quickly and the TDF supports predictability, stability, and quality with test coverage of the areas in the Data Warehouse that are most likely to change:

1. Schema tests to validate the integrity of a schema
1. Column Value tests to determine if the data value in a column matches pre-defined thresholds or literals
1. Rowcount tests to determine if the number of rows in a table over a pre-defined period of time match pre-defined thresholds or literals

The implementation details of these tests are documented in our dbt guide.

Trusted Data Dashboard

The data team is working on either a dashboard or the use of collections to organize trusted data dashboards as well as published Tableau Data Sources which are certified as trusted data.

Test Run

More to come.

Row Count Test

The row count tests reconcile the amount of rows between source database and target database by extracting data from source DB tables and load into Snowflake table and extract similar stats from Snowflake and perform comparison between source and target. There is a challenge to have an exact match between source and target, because;

- There is timing difference.
- Data warehouse might keep history.
- Deletions take place on source database.

Depending on the scenario, it's advisable to check the row count not on the highest (table)

level, but check the row counts on a lower granular level. This could be one or more fields with a logical distribution, but still on an aggregated level. An example could be an insert or update date in a table.

Based on the row counts from source and row counts on the target (Snowflake data warehouse), a reconciliation can take place to determine if all rows are loaded into the data warehouse.

Row Count Tests PGP

The framework is designed to handle execution of any kind of query to perform the test. As per the current architecture every query will create one Kubernetes pod, so grouping into one query reduces creation of the number of Kubernetes pods. For record count and data actual test between postgres DB and snowflake the approach followed is grouping low volume source tables together and large volume source tables run as an individual task.

A new yaml file is created which is supposed to do all types of reconciliation (so its not incorporated in the existing yaml extraction manifest). Manifest file combines a group of low volume tables together and a large volume table as individual tasks. Row count test comparisons from Postgres and snowflake are stored in a snowflake table named "PROD"."WORKSPACEDATA"."PGPSNOWFLAKECOUNTS".

Data Pump

mermaid
graph LR

yaml>pumps.yaml]

```
dataModel[(data model)] --> o{{Airflow DAG}}
```

```
yaml --> o  
o --> S3
```

```
S3 --> workato{{Workato recipe}} --> target[(Target)]
```

In order to make it easy for anyone to send data from Snowflake to other applications in the GitLab tech stack we have partnered with the Enterprise Applications Integration Engineering team to create this data integration framework, which we are calling Data Pump.

This is all orchestrated in the Data Pump Airflow DAG, which runs the pump, and is set to run once daily at 05:00 UTC.

Adding a Data Pump

Step 1: Create a data model using dbt in /marts/pumps (or /marts/pumpssensitive if the model contains RED or ORANGE Data), following our SQL and dbt style and documentation standards. Create an MR using dbt model changes template. Once this is merged and appears in Snowflake in PROD.PUMPS or PROD.PUMPSSENSITIVE you are ready for steps two and three.

Step 2: Add Model to pumps.yml using the 'Pump Changes' MR template with the following attributes:

model - the name of the model in dbt and snowflake
timestampcolumn - the name of the column that should be used to batch the data (or null if there is none and the table is small)
sensitive - True if this model contains sensitive data and is in the pumpssensitive directory and schema
single - True if you want to create a single file in the target location. False if multiple files can be written
stage - The name of the snowflake stage you'd like to use for the target location
owner - your (or the business DRI's) GitLab handle

Step 3: Create an issue in the platypus project using the 'change' issue template so that the Integration team can map and integrate the data into the target application.

Operational Data Pumps

Model	Target system	RF	MNPI
-----	-----	--	----
pumphashmarketingcontact	Marketo	24h	No
pumpmarketingcontact	Marketo	24h	No
pumpmarketingpremiumtoulimate	Marketo	24h	No
pumpsubscriptionproductusage	Salesforce	24h	No
pumpproductusagefreeusermetricsmonthly	Salesforce	24h	No
pumpdailydatasciencescores	Salesforce	24h	Yes
pumpchurnforecastingscores	Salesforce	24h	Yes

Data Science Data Pumps

The Daily Data Science Scores Pump and the Pump Churn Forecasting Scores Pump are two specific use-cases of the data pump, used to bring data science related data from Snowflake into S3, so that it can be picked up by Openprise and loaded into Salesforce.

The source model for the Daily Data Science Scores pump called martcrmaccountid contains a combination of PtE and PtC scores, while the Churn Forecasting Scores pump source model martcrmsubscriptionid contains scores strictly related to the Churn Forecasting model.

Marketing Data Mart to Marketo

The Email Data Mart is designed to automatically power updates to Marketo to enable creation of structured and targeted communications.

Trusted Data Model to Gainsight

The Data Model to Gainsight Pump is designed to automatically power updates to Gainsight to enable creation of visualizations, action plans, and strategies for Customer Success to help our customers succeed in their use of GitLab.

Qualtrics Mailing List Data Pump / Qualtrics SheetLoad

The Qualtrics mailing list data pump process, also known in code as Qualtrics SheetLoad, enables emails to be uploaded to Qualtrics from the data warehouse without having to be downloaded onto a team member's machine first. This process shares its name with SheetLoad because it looks through Google Sheets for files with names starting with qualtricsmailinglist. For each of the files it finds with an id column as the first column, it uploads that file to Snowflake. The resulting table is then joined with the GitLab user table to retrieve email addresses. The result is then uploaded to Qualtrics as a new mailing list.

During the process, the Google Sheet is updated to reflect the process' status. The first column's name is set to processing when the process begins, and then is set to processed when the mailing list and contacts have been uploaded to Qualtrics. Changing the column name informs the requester of the process' status, assists in debugging, and ensures that a mailing list is only created once for each spreadsheet.

The end user experience is described on the UX Qualtrics page.

Debugging to Qualtrics Processes

Attempting to reprocess a spreadsheet should usually be the first course of action when a spreadsheet has an error and there is no apparent issue with the request file itself. Reprocessing has been necessary in the past when new GitLab plan names have been added to the gitlabapiformattedcontacts dbt model, as well as when the Airflow task hangs when processing a file. This process should only be performed with coordination or under request from the owner of the spreadsheet, to ensure that they are not using any partial mailing list created by the process, as well as not making any additional changes to the spreadsheet.

To reprocess a Qualtrics Mailing List request file:

1. Disable the Qualtrics Sheetload DAG in Airflow.
2. Delete any mailing lists in Qualtrics that have been created from the erroring spreadsheet. You should be able to log into Qualtrics using the Qualtrics - API user credentials and delete the mailing list. The mailing list's name corresponds to the name of the spreadsheet file after qualtricsmailinglist., which should also be the same as the name of the tab in the spreadsheet file.
3. Edit cell A1 of the erroring file to be id.
4. Enable the Qualtrics Sheetload DAG in Airflow again and let it run, closely monitoring the Airflow task log.

*<i class="fas fa-toggle-on -text-purple"></i>*Data Spigot

A Data Spigot is a concept/methodology to give external systems, access to Snowflake data in a controlled manner. To give external systems access to Snowflake, the following controls are in place:

- A dedicated service account with a/an key-pair/OAuth authentication.
- A dedicated view (or views) only exposing the minimum required data. No Personally Identifiable Information (PII) may be disclosed.
- A dedicated role (or equivalent) with access to only the specified tables/views.
- A dedicated XS warehouse to limit and monitor costs.
- A network security policy to limit network traffic to a specific IP (range)

The process for setting up a new Data Spigot is as follows:

1. Comply to the controls that are in place, as described above.
2. Add new Data Spigots to the table below:

Current Data Spigots

Connected system	Data scope	Database table/view	MNPI
-----	-----	-----	----
Grafana	Snowplow loading times	prod.legacy.snowplowpageviewsallgrafanaspigot	No
Gainsight		prod.commonprep.prepusagepingnolicensekey	No
Gainsight		prod.commonmartproduct.martproductusagewave13metricslatest	No
Gainsight		prod.commonmartproduct.martproductusagewave13metricsmonthly	No
Gainsight		prod.commonmartproduct.martproductusagewave13metricsmonthlydiff	No
Gainsight		prod.commonmartproduct.martsaasproductusagemetricsmonthly	No
Gainsight		prod.commonmartproduct.martproductusagepaidusermetricsmonthly	No
Gainsight		prod.commonmartproduct.martproductusagefreeusermetricsmonthly	No
Gainsight		prod.restrictedsafecommonmartsales.martarr	Yes
Salesforce		martproductusagepaidusermetricsmonthly, martproductusagepaidusermetricsmonthlyreportview	No
Zapier	t.b.d.	prod.workspacecustomersuccess.martproductusagehealthscore	No

Sales Systems Use-Case: Using the Snowflake API

<i class="fas fa-clone fa-fw -text-blue"></i> Data Deduplication

Data deduplication is essential for ensuring data quality and reducing storage and compute costs in Snowflake. The current GitLab.com pipeline is designed to execute a full data extract for specific tables where incremental extraction is not feasible, as well as for tables intended for Slowly Changing Dimensions (SCD) modeling. To check for any missing transactions in the source system, incremental extraction tables consistently overlap by 30 minutes.

Additionally, all data sourced from another application, CustomersDot, is extracted in full twice a day, as each extract plays a role in building the SCD downstream.

To address our need for reduced Service Level Objectives (SLO) and Service Level Agreements (SLA), we have shifted towards more frequent extracts for both CustomersDot and GitLab.com. This adjustment has resulted in an increase in duplicate records and higher storage requirements in Snowflake for tables associated with both full and incremental extracts. The growing number of duplicates has adversely affected the results of the dbt model and dbt tests on these data sources over time.

To decrease dbt runtime and enhance the efficiency of Snowflake's computing and storage, we developed a deduplication framework specifically targeting these data sources. This framework can be easily extended to other data sources in Snowflake where duplicate records may accumulate.

Deduplication Framework

The deduplication framework consists of two main components:

1. Airflow: Airflow consists of 3 deduplication DAG's:

- i. Deduplication DAG for gitlab.com incremental extract tdeduplicationgitlabcomincremental
- ii. Deduplication Staging DAG for gitlab.com scd (full) extract tdeduplicationgitlabdbscd
- iii. Deduplication SCD DAG for CusotmerDot SCD extract.tgitlabcustomersdbdbt

Since we maintain the list of the tables, we extract data in the manifest file as part of gitabdataextract pipeline. Airflow relies on the exact source of truth to get the list of the tables for which it has to run the deduplication logic. The DAG is scheduled to run weekly.

2. Snowflake: In Snowflake, the following activities are carried out:

- i. Backup tables are created using Snowflake clone command with timestamp suffixes in the TAPPOSTGRESBKP schema inside of the RAW database.
- ii. A temporary table is created with a deduplicated dataset using a GROUP BY clause to eliminate duplicates while retaining the most recent records and managing special columns like uploadedat and taskinstance. The deduplication logic selects all unique rows from the table.
- iii. The temporary tables are swapped with the original tables, while maintaining current grants and permissions.
- iv. Temporary tables are dropped after a successful swap.
- v. Delete the backup table older than 7 days.

Visualization

We use Tableau as our Data Visualization and Business Intelligence tool. To request access, please follow submit an access request. Use the template TableauRquest for Tableau access requests.

Meta Analyses for the Data Team

Tableau Usage! · - coming soon

Tableau Account Optimization · - coming soon

Tableau Account Maintenance · · - coming soon

dbt Event Logging - coming soon

Snowflake Spend · ·

Security

Passwords

Per GitLab's password policy, we rotate service accounts that authenticate only via passwords every 90 days. A record of systems changed and where those passwords were updated is kept in this Google Sheet.

We also rotate Snowflake user passwords the first Sunday of every 3rd month of the year (January, April, July, October) via the Snowflake Password Reset DAG.

Software User Provisioning

The data team is responsible for provisioning users within the tools managed by the Data Team. This includes tools like Tableau, MonteCarlo, Fivetran, Stitch, and Snowflake.

For Snowflake, we have a robust process documented in the Snowflake Permissions Paradigm section of this page.

For other tools, add users via the UI and in the appropriate Google Group if one exists.

Stitch provisioning

A new user in Stitch should by default be added to the General role. This role gives sufficient access to Stitch to create new, change existing and troubleshoot running extractions. Stitch provisioning is a two-step process. First, the IT operations team adds the team member to the app.stitch Okta group by feeling the Access Request. The second step involves adding the user's email to the Stitch application.

Google Data Studio

Much like Google Drive all GitLab team members have access to Google's Data Studio which can be used to build dashboards with data from Google Sheets or other Google data sources. Hence there is no access request needed to get access provisioned to Google Data Studio.

Google Data Studio is especially popular with Marketing with their use of Google Analytics.

Though this resides outside of the platform described above, any data managed within Google's Data Studio must adhere to the same Data Categorization and Management Policies as we do in the rest of our platform.

There are 3 types of objects available in Google Data Studio:

Data Sources

- This is a connection to data sources. Currently there is no connection available/supported towards our Snowflake data warehouse.

Reports

This is for creating reports based on any connected data set.

Explorer

This is a tool to quickly explore data sets and find detailed insights.

The sharing and access process in Data Studio is comparable to sharing in Google Drive / Google Docs. Google Studio Objects can be shared with individuals in our GitLab organization account or with the Organization as a whole. There are no group or role level permissions available. Given the decentralized quality of managing dashboards and data sources in Data studio it is advised that business critical data and reporting be eventually migrated to Snowflake and Tableau. This is made easy with the use of sheetload or FiveTran, which has a BigQuery connector.

A GitLab Team Member that creates any artifacts in Google Studio owns the owner permissions of that particular object. With the ownership the GitLab Team Member holds responsibility to keep data SAFE within GitLab and outside the organization. Google Data Studio currently doesn't provide an admin interface that can take over the ownership. Upon off-boarding any ownership of existing objects should be carried over to ensure business continuity by the respective object owner. Note that Red Data should never be stored or transmitted within Google Data Studio.

Sales Analytics Notebooks

The Sales Analytics have a couple (but expanding) list of regular update processes that will benefit from being able to be run automatically without human intervention.

Some of those are:

1. X-Ray fitted curves calculation: Quarterly process that create a table with fitted curves to historical coverage ratios. This data is used within the X-Ray dashboard.
2. QTD Pre-Aggregated data for X-Ray and SAE Heatmap: Daily process to precalculate data aggregations at different levels. This process is much easier to run in Python than with SQL and we will be able to upload the data directly into Snowflake.

For this, we have implemented a solution consisting of multiple Airflow dags, per schedule.

The process, explained

As of right now (subject to further iterations and changes), the steps are the following:

1. A Sales Analyst works on a Python Notebook (example notebook) and makes it ready for production (making sure the cell execution results are cleared, no local variables/secrets are laying around etc.)
2. The Sales Analyst uploads the notebook and its respective query in the corresponding folder, depending on what schedule the notebook should run on. The available schedules (and therefore folders) under <https://gitlab.com/gitlab-data/analytics/-/tree/master/salesanalyticsnotebooks> are:

- daily - daily at 6AM
- weekly - every Monday at 6AM
- monthly - every 7th day of the month, at 6AM
- quarterly - every 7th day of the quarter, at 6AM

This has been implemented by creating 4 main DAGs (one per schedule) consisting of as many tasks as there are notebooks for that schedule. New tasks are dynamically added to the DAG as notebooks are committed to the repository.

The code for the dags can be found in the Sales Analytics Dags in the gitlab-data/analytics project.

Example

Currently, under the /daily/ notebooks we have one sample notebook and its corresponding query.

This notebook runs daily and the scores produced during execution are loaded into Snowflake in the RAW.SALESANALYTICS schema.

For this data to be available on Tableau, dbt models will have to be written to expose them as views in the production database, under the PROD.WORKSPACESALES schema.

For that the Sales Analyst can either open an MR directly into the gitlab-data/analytics project, or create an issue on this project and a data platform engineer will implement the necessary dbt models.

In order to change the desired day of the week/time of these schedules, the Sales Analyst can open an issue on the [gitlab-data/analytics](#) project.

Failure notifications

Dag failure alerts are sent from Airflow to the sales-analytics-pipelines, so the Sales Analysts can monitor errors with the notebooks

If the errors seem to be platform-related, the Sales Analyst can reach out to the data platform engineers either via Slack (via the data-engineering channel), or by opening an issue on the [gitlab-data/analytics](#) project

GSheets & Jupyter Notebooks

A couple of new functions have been added to the [GitLabdata](#) library ([Link to PyPi](#), [Link to the source code](#)) to allow reading from and writing to GSheets files.

Reading from GSheets within Jupyter Notebooks

The function is called `readfromgsheets`([link to function source code](#)) and it accepts a `spreadsheetid` and a `sheetname` as parameters, it returns a dataframe.

> :warning: The specific sheet should be shared with the relevant gCloud SERVICE ACCOUNT user's email account (See System Set Up).

Writing to GSheets from Jupyter Notebooks

The function is called `writetogsheets`([link to function source code](#)) and it accepts a `spreadsheetid`, a `sheetname` and a dataframe as parameters.

> :warning: The specific sheet should be shared with the relevant gCloud SERVICE ACCOUNT user's email account (See System Set Up).

System set up - Remote execution

For production use-cases, a service user has been provided and the credentials are stored in the Data Team Secure Vault under GCP Service Account for Exporting to GSheets.

> :warning: The specific sheet should be shared with the service account user's email (`data-team-sheets-sa@gitlab-analysis.iam.gserviceaccount.com`) prior to calling this function, otherwise the account won't be able to write to or read from the sheet.

System set up - Local / Team development

For local development, you need to set the `GSHEETSSERVICEACCOUNTCREDENTIALS` environment variable with the value of your team's gCloud SERVICE ACCOUNT Credential (The actual JSON content should be the value of this environment variable, not the path.)

This can be done by running the following command in the terminal of your choice: `export GSHEETSSERVICEACCOUNTCREDENTIALS = 'JSONCREDENTIALCONTENTS'`

To maintain our high standard in security and avoid any potential breaches, it is required that each team requests and manages their own gCloud SERVICE ACCOUNT.

The GCP team can support with the creation of the user / GCP project. Here is an example of the issue to create the Service Account for the Revenue Strategy and Analytics team.

The gCloud SERVICE ACCOUNT requires Google Workspace Delegated Admin permissions.

> :warning: The specific sheet should be shared with the gCloud SERVICE ACCOUNT user's email prior to calling this function, otherwise the account won't be able to write to or read from the sheet.

Remaining work

Update the repository URL for the sales analytics notebooks (link to issue)

Sales Systems Use-Case: Using the Snowflake API

The Sales Systems team needs to run the same query several times per day against Snowflake and load that data into Salesforce.

The data team provided an API user so the Sales Analytics team can automate this process, instead of manually downloading the data and uploading it into Salesforce. More detail on this use-case can be found in the original issue 15456.

The data pulled from the database is encapsulated in a view that strictly exposes only the requested data and the sales systems team will be querying this view directly via the Snowflake API.

A new role was created specifically, called SALESSYSTEMSSNOWFLAKEAPIROLE for this use-case on Snowflake and it has been configured to only have read access on the underlying view.

The Snowflake API user has been created following the steps in the official Snowflake documentation on Using Key Pair Authentication and the credential is stored in our Data Team Secure vault and is to be shared with the Sales Systems team.

We created a runbook with a step-by-step guide on how to create the user and role for this purpose - link to the Snowflake API User runbook.

Exceptions

Exceptions to this standard will be tracked as per the Information Security Policy Exception Management Process.

References

The platform infrastructure

SECTION: enterprise-data/platform/aws-guide.md

Context

The data platform team uses AWS to support:

- Snowplow

Furthermore, other pipelines use S3 buckets to store data, such as:

- just global
- greenhouse

Getting access to AWS

- An AR requesting rolename DataEngineeringAccess should be filled out, more details in this issue comment.

awscli

Occasionally, you will need to do actions via the command line. For example, if you would like to do query a file in S3, but it exceeds the file size limit to use the UI's 'S3 Select', you will have to query it from the command line instead.

Installing awscli

In the 'Onboarding template', you should have already installed awscli. If not, install it now.

Authenticating with awscli

1. Via Okta, go to your AWS access portal
1. For gitlab SaaS production account, next to DataEngineeringAccess role, click Access keys
1. Copy the following information
 - SSO start URL
 - SSO Region
1. In the command line, run `aws configure sso`
 1. This will prompt you to enter the information you saved above
 1. Then, a pop-up will enter in your internet browser asking you confirm
 1. Lastly, in your shell, you will need to select/enter the following:
 - choose the project, i.e gitlab SaaS production
 - choose the role, i.e DataEngineeringAccess
 - For the remaining prompts, i.e region, you can just type `\<enter>` to use the default value
1. After authentication, you can run AWS CLI commands as usual by adding the `--profile DataEngineeringAccess-8552xxxxx` argument. The CLI prompt will remind you to include this.

Example command

A 'S3 select' command using the cli:

```
sh
aws s3api select-object-content \
```

```
--bucket datateam-just-global-campaigns \  
--key datateam-just-global-campaignsInfo/2024-12-19-GitLabAnalyticsExportJG-  
from-2023-07-01-until-2024-12-15.csv \  
--expression "SELECT FROM s3object LIMIT 5" \  
--expression-type 'SQL' \  
--input-serialization '{"CSV": {"FileHeaderInfo": "NONE"}}' \  
--output-serialization '{"CSV": {}}' \  
--profile DataEngineeringAccess-8552xxx \  
output.csv
```

SECTION: enterprise-data/platform/dbt-guide.md

Quick Links

- Primary Project
- dbt docs

What and why

dbt, short for data build tool, is an open source project for managing data transformations in a data warehouse. Once data is loaded into a warehouse, dbt enables teams to manage all data transformations required for driving analytics. It also comes with built in testing and documentation so we can have a high level of confidence in the tables we're generating and analyzing.

The following links will give you an excellent overview of what dbt is:

- What, exactly, is dbt? - This is a less technical overview for understanding the tool
- What is dbt? - This is a bit more technical and comes straight from the docs

But why do we use dbt? There are several reasons.

First is that it is an open source tool with a vibrant community.

Choosing an open source tool enables us to collaborate with the larger data community and solve problems faster than had we gone with a proprietary solution.

Second, it was built with version control in mind.

For GitLab, this is essential since we use the product for building and running the company.

Third, it speaks the language of analysts - SQL.

This increases the number of people that can contribute since SQL is becoming such a critical part of many people's jobs.

Finally, it enables teams to move faster by integrating testing and documentation from the start.

For even more information about the basics of dbt, see our data analyst onboarding issue template

At times, we rely on dbt packages for some data transformation.
Package management is built-in to dbt.
A full list of packages available are on the dbt Hub site.

Running dbt

If you're interested in using dbt, the dbt documentation has a great tutorial on getting setup to work on data from a fictional business called Jaffle Shop.

If you wish to use dbt and contribute to the data team project, you'll need to gain access to our Snowflake instance, which can be done via an access request.

Local environment

We use user-dedicated development databases and virtual environments via make recipes in order to facilitate a 'local' development environment for GitLab team members contributing to our dbt project(s)

'Local' User-Dedicated Development Databases

When needed for team members we create local development databases corresponding to the snowflake user with PREP and PROD suffixes, corresponding to the PREP and PROD databases in snowflake. These will be targeted by our main dbt project when run within the local environment so that contributors can develop and test changes to our dbt project. More detail on our development process within our dbt project can be found on the dbt Change Workflow page.

Any data built within these development databases should be considered ephemeral as they're only to be used for local development. To ensure the optimal use of dbt, as well as appropriate security and compliance, these databases should be cleaned by the owning user regularly. This Runbook can be used to make that process quick and easy, and it's suggested to be run at the end or beginning of each development cycle. Additionally, in order to ensure compliance with our data retention policies and procedures we will automatically drop all tables in development environments after 80 days without alteration. This retention period is set within the dbt project with the `devdbobjectexpiration` variable and tables are deleted each weekend.

Note: Development databases are dropped as soon as the corresponding Team Member is deprovisioned access to Snowflake (i.e. in case of offboarding or inactive usage. There is not [backup]/handbook/enterprise-data/platform/backups) process for development databases.

Configuration

- Ensure you have access to our Snowflake instance and dedicated development databases for your snowflake user
- Ensure you have Make) installed (should be installed on new Macs and with XCode)
- Create a folder in your home directory called `.dbt`
- In the `~/dbt/` folder there should be a `profiles.yml` file that looks like this sample profile
- The smallest possible warehouse should be stored as an environment variable. Our dbt jobs use `SNOWFLAKETRANSFORMWAREHOUSE` as the variable name to identify the warehouse. The environment variable can be set in the `.bashrc` or `.zshrc` file as follows:
 - `export SNOWFLAKETRANSFORMWAREHOUSE="DEVXS"`

- In cases where more compute is required, this variable can be overwritten at run. We will cover how to do this in the next section.
- Clone the analytics project
- If running on Linux:
 - Ensure you have Rancher Desktop Installed

Note that many of these steps are done in the onboarding script we recommend new analysts run.

Choosing the right Snowflake warehouse when running dbt

Our Snowflake instance contains warehouses of multiple sizes, which allow for dbt developers to allocate

differing levels of compute resources to the queries they run. The larger a warehouse is and the longer it runs, the more the query costs. For example, it costs 8 times more to run a Large warehouse for an hour than it costs to run an X-Small warehouse for an hour.

If you have access to multiple warehouses, you can create an entry for each warehouse in your profiles.yml file. Having done this, you will be able to specify which warehouse should run when you call dbt run. This should be done carefully; using a larger warehouse increases performance but will greatly increase cost! Err on the side of using smaller warehouses. If you find that the smaller warehouse's performance is not adequate, investigate the cause before you try again with a larger warehouse. Running an inefficient model against a larger warehouse not only increases cost during development, it also increases cost every time the model runs in production, resulting in unintentional ongoing increase in Snowflake costs.

Example

Imagine that you are a Data Team member who needs to make a change to a dbt model. You have access to both an X-Small warehouse and a Large warehouse, and your profiles.yml file is set up like so:

```
yml
gitlab-snowflake:
  target: dev
  outputs:
    dev:
      type: snowflake
      the
```